

CS342 OPERATING SYSTEMS

Spring 2026 - Project #4

MY FUSE FILE SYSTEM (MFS)

Assigned: April 19, 2026

Due date: May 8, 2026; 23:59.

Document Version: **1.0**

- The project will be done in groups of 3 students at most. You can do it individually if you wish.
 - You will program in C/Linux. Programs will be tested in Ubuntu 24.04 Linux 64-bit.
 - You are not allowed to post cs342 project/homework/exam assignments and/or their solutions to public places such as github. This is very important. This rule applies to graduated students as well.
 - You are not allowed to look at a solution from Internet, from other sources, from previous semesters, or at the code of your friend. You should come up with your own solution and implementation. You can use your previous solutions for standard data structures like lists.
 - **Objectives/keywords:** Practice with file systems, FUSE framework, block allocation, callbacks, mounting/unmounting.
-

In this project you will design and develop a simple **filesystem** that is using the **FUSE** framework [1]. The name of the filesystem will be **MFS** and it will be implemented in C, in a file called `mfs.c`. The **mfs** program, hence the file system, will run in **user-space**. It will store files and meta-data in a regular Linux file. This Linux file will be the **disk** of the file system.

FUSE (Filesystem in Userspace) framework allows us to implement and run a filesystem in user-space. Its components include a Linux kernel module and a Linux library [2]. With FUSE we can develop and integrate a user-level filesystem without knowing kernel development. A Linux kernel with FUSE extension will direct the file-related system calls to our user-level filesystem process via **callbacks**. Our file system will handle the file-related requests.

You will use **FUSE version 3 (not version 2)**. To install the development package for FUSE 3, you can run the following command (if it is not already installed in the system): `sudo apt install libfuse3-dev`. You can get help from Internet to learn about proper installation and compilation. You must use the **-DFUSE_USE_VERSION=31** for compiling (it has to be 31; this is what we will use in compiling and testing your programs). See the Makefile that is provided.

MFS FEATURES

MFS will be a simple file system. It will use **indexed** allocation method to keep track of disk blocks allocated to files. It will use bitmap to keep track of free blocks. The file system will have only a single directory, which is the root directory. All files will be stored in the root directory. There will be no subdirectories. The file system will use **fixed** size **filenames** and fixed size directory entries.

For a file, the respective directory entry will contain the inode number of the inode (FCB) of the file.

For each file, there will be an inode (FCB). The **inode** for a file should contain meta-data about the file, i.e., file attributes and location of file data. This information may include the size of the file in bytes, the number of disk blocks allocated for the file, the inode number, and the block number of the index block for the file (i.e, data block pointer), etc. An inode can keep additional information if you wish, like file creation and modification times, permission bits, etc., but these are not mandatory.

The **index block** for a file will store data block pointers (i.e., data block numbers) of the data blocks allocated to the file (for file content/data). There will be exactly **one** index block used for a file. Hence single-level index structure is used. For each data block allocated to the file, there will be one pointer to the data block.

The design **parameters** of the file system will be as follows:

- Block size is 16 KB (2^{14} bytes).
- The first block of the disk (block 0) will contain superblock information (volume information).
- The next block (block 1) will contain the bitmap for free space management.
- The next block (block 2) will contain the inode map.
- The next two blocks will contain the inode table.
- The next 1 block will contain the root directory.
- Filenames will be of fixed size. A filename can be at most 31 characters long. We need to store the NULL character at the end of a filename. Hence together with the NULL character, a filename can be at most 32 characters long.
- A directory entry will be of fixed size. Directory entry size is 64 bytes.
- Inode size is 128 bytes. All inodes are of the same size.
- A single disk block can contain $16 \text{ KB} / 128 \text{ bytes} = 128$ inodes.
- A single disk block can contain $16 \text{ KB} / 64 \text{ bytes} = 256$ directory entries.
- The number of inodes in the inode table is $2 * 128 = 256$.
- The **maximum number of files** in the file system can be at most 254. We will have 256 directory entries in the root directory, but two of these entries will be used for special entries "." and ".." (that will be mentioned later). Hence: $256 - 2 = 254$ new files.
- Inodes are numbered starting with 1. Inode 1 will be for the root directory. Hence a file on the disk can have an inode number that is ≥ 2 .

- We consider the blocks of the disk are **numbered** sequentially as 0, 1, 2, ..., $N - 1$ (N indicates the number of blocks on the disk). The number of blocks on a disk will be equal to the disk size divided by the block size. We can call disk blocks also as logical blocks (LBs), or logical disk blocks.
- Disk pointer (disk address) size is 4 bytes. A pointer value is simply a **disk block number**, i.e., a logical block address (LBA).
- An inode will have no direct data block pointers. It will just point to an index block for the file. There will be **one index block** for a file.
- A disk block that contains data block addresses for a file is called an **index block**. An index block can contain $16 \text{ KB} / 4 \text{ bytes} = 4096$ block numbers (addresses). That means 4K addresses.
- Hence **maximum size of a file** can be $4096 * 16 \text{ KB} = 64 \text{ MB}$.
- In the bitmap we have $2^{14} * 2^3 = 2^{17} = 128 \text{ K}$ bits. Hence we can have 128K disk blocks at most in the disk. Hence **max disk size** can be **2 GB**.

A **bitmap** will be used to keep track of the free blocks on the disk. For each block of the disk, there will be a bit in the bitmap indicating if the block is used or not. 1 means used; 0 means free.

The **inode-map** will be used to keep track of the allocated and free inodes. Initially, only the root inode is used. All other inodes are free. 1 means used; 0 means free.

Figure 1 shows the MFS **layout** on a **disk**.

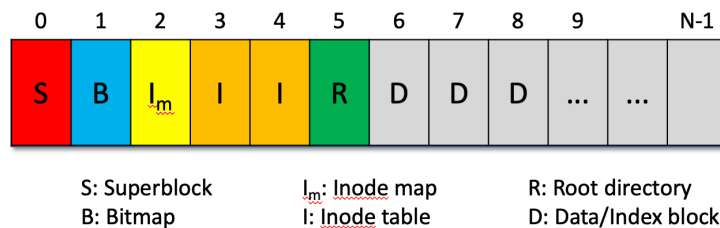


Figure 1. Disk layout of MFS. First block (block 0) contains superblock information. Assume there are N disk blocks. Inode Table occupies two blocks (blocks 3 and 4).

Figure 2 shows the logical place of the `mfs` program in a Linux system. It is **linked** with `libfuse3` library and `libc` library. An application program (app) can use the file system by creating, opening, reading, writing files. An application is linked with the `libc` library so that the related file I/O routines called by the application can be converted to the system calls issued to the operating system kernel. **VFS** layer in the kernel directs the operations to the **FUSE module** in the kernel. The FUSE module calls the related **handler functions** (callback functions) implemented in `mfs`.

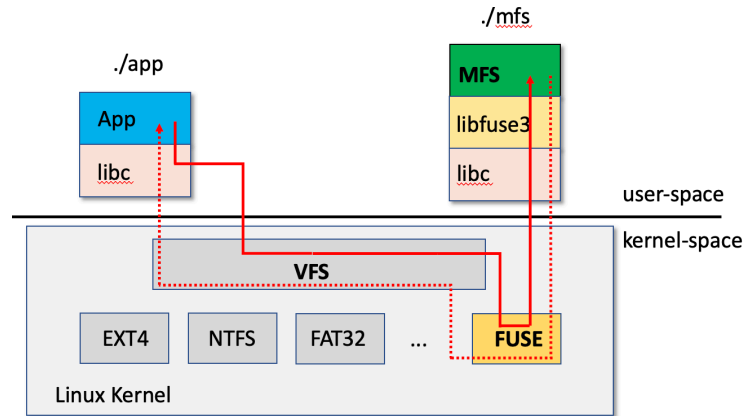


Figure 2. An application can issue file related systems calls for files on the MFS controlled disk. The calls are directed to MFS by the kernel (by VFS and FUSE modules in the kernel).

OPERATIONS TO SUPPORT

Your MFS file system will support the following **operations**. That means a user program (an application) should be able to do the following actions on the file system.

- List the files in the current directory.
- Create a file.
- Delete a file.
- Get attributes of a file.
- Open a file.
- Close a file (release).
- Read from a file (when an application calls the read system call).
- Append into a file (when an application calls the write system call).

CREATING THE DISK FILE

We can create a regular Linux file to act as a **disk** that will be used by our file system. Assume the name of our disk file is `disk1`. Assume we want the size of the disk to be 16 MB. Since the block size is 16 KB, we will have $16 \text{ MB} / 16 \text{ KB} = 2^{10} = 1024$ blocks on the disk. We can create the disk file with **dd** command.

```
$ dd bs=16K count=1K if=/dev/zero of=disk1
```

We need to do this only once, or whenever we want to completely destroy the disk.

INITIALIZING THE FILE SYSTEM (FORMATTING)

You also need to write a program called **make_mfs** (`make_mfs.c`) that will initialize (i.e., **format**) the disk with the MFS file system. The on-disk data structures (superblock, ...) will be created and **initialized** on the disk. Initially there will be no file on the disk, except two special files "." and "..". Therefore, initially, when we type `ls` in the root directory of the MFS file system, only two entries should be listed: "." and "..". As a Unix user, you should know what they mean. This means that initially (at file system formatting time) the root directory will just contain two entries.

Since two entries of the root directory will be used for "." and "..", we can use the remaining entries for files that will be created. Hence, **maximum number of files** that can be created is: $256 - 2 = 254$.

USING FUSE TO IMPLEMENT FILE SYSTEM

There is a lot of documentation and examples in Internet about how to **write a filesystem** at userspace by using the FUSE framework. You can search for and benefit from them. Basically, you need to write the related handler functions and put their addresses into a structure of type "struct fuse_operations". This structure is defined in the header file of the fuse library: `/usr/include/fuse3/fuse.h`. You can examine the content of that header file as well.

You don't need to implement all the operations specified in the file operations structure in the header file. Only some of them will be implemented. You can learn more about these operations via their comments in the header file `fuse3/fuse.h`. There is also a lot of information on Internet and in libfuse github project.

In your program you will define and use a file operations structure. You will initialize its fields with the addresses of the implemented functions. For example, below we define a structure `mfs_oper` of type `struct fuse_operations` and initialize it.

```
struct fuse_operations mfs_oper = {
    .getattr  = mfs _getattr,
    .readdir  = mfs _readdir,
    .open     = mfs _open,
    .read     = mfs _read,
    .release  = mfs _release,
    .write    = mfs _write,
    ....
};
```

The functions `mfs_getattr`, `mfs_readdir`, `mfs_read`, etc., are the functions that you need to implement according to the specified function prototypes in the operations structure.

The `main()` function of your program should include a call to `fuse_main()` function to **register** the operations to the kernel and to **mount** the file system.

```

int main(int argc, char *argv[]) {
    ...
    ...
    // mount the file system
    fuse_main( argc, argv, &mfs_oper, NULL);

    return 0;
}

```

Your program will be started as:

```
./mfs /tmp/fusemountpoint diskfilename
```

Hence, in the program, `argv[0]` will be the name of the program and `argv[1]` will be the mount point (i.e., `/tmp/fusemountpoint` in the example).

You can also examine and run the example `hello.c` program that you can find in the **libfuse project** in github [1]. The project directory has other examples as well.

Hint: In the `main()` function of `mfs.c`, after getting the `diskfilename` argument from command line (via `argv` array) and storing it in another character array variable (a string variable), you may need to delete that argument (i.e., `diskfilename`) from the `argv` array before calling `fuse_main()` function. Otherwise, `fuse_main()` may think that there are two mount points given, and therefore may not mount properly.

BUILDING THE FILE SYSTEM

We can compile our file system code with a Makefile. The following is a sample Makefile to do this. It will generate the `mfs` program. Before running `mfs`, we should have our disk created and initialized (formatted) with MFS.

```

all: mfs make_mfs

mfs: mfs.c
    gcc -O2 -Wall -D_FILE_OFFSET_BITS=64 -DFUSE_USE_VERSION=31 -c -
o mfs.o mfs.c
    gcc mfs.o -lfuse3 -O2 -Wall -D_FILE_OFFSET_BITS=64 -
DFUSE_USE_VERSION=31 -o mfs

make_mfs: make_mfs.c
    gcc -Wall -g -D_FILE_OFFSET_BITS=64 make_mfs.c -o make_mfs

clean:
    rm -f *~ mfs.o mfs make_mfs

```

MOUNTING AND UNMOUNTING THE FILE SYSTEM

We can **mount** our FUSE file system by running the `mfs` program. Assume we created an empty directory in `/tmp` that will act as our mount point in our default directory tree. Assume the name of the mount point is `"/tmp/fusemountpoint"` (we could give another name). Then, to start and mount our file system, we type:

```
$ ./mfs /tmp/fusemountpoint/ diskfilename
```

When the program starts, the MFS file system on the disk is **mounted** at the mount point and the file system is ready to use. Its functions are **registered** to the kernel. It is running; waiting for a file I/O related event to happen. VFS layer in the kernel will direct the calls to the FUSE kernel module, which will call our registered handler functions.

For example, we can change into the mounted directory and run the `ls` program to list the files there.

```
$ cd /tmp/fusemountpoint/  
$ ls
```

Then we should see the list of files that exist in the directory `/tmp/fusemountpoint/`.

To **unmount** the filesystem we type:

```
$ fusermount -u /tmp/fusemountpoint
```

We should unmount when we are done using the filesystem, or when we have modified and recompiled our filesystem code.

APPLICATIONS

We can write an application that will use files in the mounted directory. It can, for example, open an existing file in `/tmp/fusemountpoint/` and read data from it, as in the following pseudo-code.

```
fd = open ("/tmp/fusemountpoint/...", ...);  
read (fd, ..., ...);
```

ACCESSING THE DISK FILE IN YOUR MFS PROGRAM

In your `mfs` program, you should access the disk **block by block**, in other words, one block at a time. For that you will use two operations: `read_block()` and `write_block()`. Even though you would write (or read) a single byte, you need to write (or read) the whole respective block on

(from) the disk. A possible implementation of `read_block()` and `write_block()` functions is shown below.

```
#include <sys/types.h>
#include <unistd.h>

#define BLOCKSIZE 16384 // bytes - it makes 16 KB

// Read block k from disk (Linux file) into buffer block
// block is the buffer (a character array).
// k is block number.
// fd_disk is file descriptor for the disk file.
// Memory space for block must be allocated outside of this function.
int read_block (int fd_disk, void *block, int k)
{
    int n;
    int offset;

    offset = k * BLOCKSIZE;
    lseek(fd_disk, (off_t) offset, SEEK_SET);
    n = read (fd_disk, block, BLOCKSIZE);
    if (n != BLOCKSIZE) {
        printf ("read error\n");
        return -1;
    }
    return (0);
}

// write block k into the disk.
int write_block (int fd_disk, void *block, int k)
{
    int n;
    int offset;

    offset = k * BLOCKSIZE;
    lseek(fd_disk, (off_t) offset, SEEK_SET);
    n = write (fd_disk, block, BLOCKSIZE);
    if (n != BLOCKSIZE) {
        printf ("write error\n");
        return (-1);
    }
    return 0;
}
```

EXPERIMENTS AND REPORT

You will design and conduct some **performance** (time) experiments. You will then interpret the results. You will put your results, interpretations, and conclusions in a `report.pdf` file. You will include this file as part of your uploaded package. Experiments and the related report will be **20%** of the project grade.

SUBMISSION

Put all your files into a directory named with your Student ID. If the project is done as a group, the IDs of both students will be written, separated by a dash '-'. In a README.txt file, write your name, ID, etc. (if done as a group, all names and IDs will be included). The set of files in the directory will include README.txt, Makefile, and program source file(s). We should be able to compile your program(s) by just typing make. No binary files (executable files or .o files) will be included in your submission. Then **tar** and **gzip** the directory, and submit it to **Moodle**.

For example, a project group with student IDs 21404312 and 214052104 will create a directory named 21404312-214052104 and will put their files there. Then, they will tar the directory (package the directory) as follows:

```
tar cvf 21404312-214052104.tar 21404312-214052104
```

Then they will gzip (compress) the tar file as follows:

```
gzip 21404312-214052104.tar
```

In this way they will obtain a file called 21404312-214052104.tar.gz. Then they will upload this file to **Moodle**. For a project done individually, just the ID of the student will be used as a file or directory name.

LATE POLICY FOR THE PROJECT

If the deadline is missed by up to 24 hours, 25 points will be cut from the graded project and that will be the grade of the project. For example, if a late submission is graded and received 80 pts; then the grade of the project will be $80 - 25 = 55$. If the deadline is missed between 24 - 48 hours, then 50 pts will be cut. Submission beyond 2 days (48 hours) will not be allowed. Note that even the submission is 1 second later after the deadline, 25 points will be cut. System assigns timestamps to the submissions. We will use those timestamps. Therefore, you are recommended to finish your submission 2-3 hours (at the latest) before the deadline. In fact, it is better if you submit one day before. You can re-submit as many times as you wish by the deadline. Each submission will overwrite the previous one. Please do not send your submission with email. Submission system will be open beyond the deadline up to 48 hours.

TIPS AND CLARIFICATIONS

- Start early, work incrementally.
- A filename can include letters (lowercase, uppercase), digits, and the following special characters: dot character (ascii code: 0x2E), hyphen-minus character (ascii code:0x2D), and underscore character (ascii code:0x5F).
- For this project, assume disk size can not be bigger than 64 MB, and can not be smaller than 2 MB.
- After updating the disk and unmounting, if we mount again, we should be able to see the previous updates (like created files, modified files, etc.).

- If we create a disk of size N blocks (blocks 0, ..., N-1) with dd command, the first block (block 0) will contain the superblock information.
- If you want to unmount, you need to unmount when mfs process has terminated (or killed) and you are not in the mount directory.
- The disk file will be in the same directory as the mfs program. So, there is no need to specify an absolute path.
- An initial project skeleton (including a Makefile) will be added to github at the following address:
<https://github.com/korpeoglu/cs342spring2026-p4.git>
- The project skelaton contains a Makefile which you can use to compile your project. A copy of the fuse.h header file is also placed into the project skelaton directory for you to read it. But don't use it (with #include) in your program. Use the one in the default place for header files:
/usr/include/...
- We will setup our machine (Ubuntu 24) with the fuse3 library to test your programs.

REREFENCES

[1]. FUSE Project, "libfuse," [Online]. Available:
<https://github.com/libfuse/libfuse>. [Accessed 2026].

[2]. Wikipedia, "File System in Userspace," [Online]. Available:
https://en.wikipedia.org/wiki/Filesystem_in_Userspace. [Accessed 2026].